

Variables in C Language

1. Definition

- A **variable** is a named memory location used to store data.
- Its value can change during program execution.

2. Rules for Naming Variables

- Can contain alphabets, digits, and underscore _.
- Must begin with an alphabet or underscore.
- Cannot use reserved keywords (e.g., int, float, return).
- Case-sensitive (Age and age are different variables).

3. Declaration & Initialization

- **Declaration:** Tells the compiler the type of variable.
`int age; // declaration`
- **Initialization:** Assigns a value to the variable.
`int age = 20; // declaration + initialization`

4. Data Types & Size

Data Type	Typical Size (bytes)	Example
int	2 or 4	<code>int x = 10;</code>
float	4	<code>float pi = 3.14;</code>
double	8	<code>double d = 12.345;</code>
char	1	<code>char c = 'A';</code>

Keywords

Reserved words that have special meaning to the compiler

Program Structure

```
#include<stdio.h>

int main() {
printf("Hello World");
return 0;}
```



Comments

Lines that are not part of program

Single Line //

Multiple Line /*

Output

```
printf(" Hello World ");
```

```
printf(" kuch bhi \n");
```

Output

1.integers

```
printf(" age is %d ", age);
```

2. real numbers

```
printf(" value of pi is %f ", pi);
```

3. characters

```
printf(" star looks like this %c ", star);
```

Input

```
scanf(" %d ", &age);
```

Practice Qs 1 (Area of Square) side × side

Practice Qs 2 (Area of Circle) πr^2



Operators in C Language

Definition

Operators are special symbols in C that perform operations on variables and values. They are used to build expressions and manipulate data.

Types of Operators (with names)

Category	Operators (Names)	Symbols	Example
Arithmetic Operators	Addition, Subtraction, Multiplication, Division, Modulus	+ - * / %	a + b
Relational Operators	Equal to, Not equal to, Greater than, Less than, Greater than or equal to, Less than or equal to	== != > < >= <=	a > b
Logical Operators	Logical AND, Logical OR, Logical NOT	&& !	a && b
Bitwise Operators	Bitwise AND, Bitwise OR, Bitwise XOR, Bitwise NOT, Left shift, Right shift	& ^ ~ << >>	a & b
Assignment Operators	Assignment, Add and assign, Subtract and assign, Multiply and assign, Divide and assign, Modulus and assign, Left shift and assign, Right shift and assign, AND assign, XOR assign, OR assign	= += -= *= /= %= <<= >>= &= ^= =	a += 5
Increment/Decrement Operators	Increment, Decrement	++ --	a++

Operator Precedence and Associativity

Precedence Level	Operators	Associativity
Highest	() [] -> .	Left to Right
Multiplicative	* / %	Left to Right
Additive	+ -	Left to Right
Shift	<< >>	Left to Right
Relational	< <= > >=	Left to Right
Equality	== !=	Left to Right
Bitwise AND	&	Left to Right



Precedence Level	Operators	Associativity
Logical AND	&&	Left to Right
Logical OR		Left to Right
Assignment	= += -= *= /= %= <<= >>= &= ^= =	Right to Left
Comma	,	Left to Right

Example:

```
int a = 6, b = 3, c = 4;
int res = a + b * c / 2;
```

Here, * and / are evaluated before +, so:

$6 + ((3 * 4) / 2) = 12.$

Conditional Statements in C Language

Definition

Conditional statements are used to make decisions in a program. They allow the program to execute different blocks of code depending on whether a condition is **true** or **false**.

Types of Conditional Statements

1. if statement

- Executes a block only if the condition is true.

```
if (x > 0) {
    printf("Positive number");
}
```

Example: Prints "Positive number" if x is greater than 0.

2. if-else statement

- Executes one block if the condition is true, otherwise executes another block.

```
if (x % 2 == 0) {
    printf("Even number");
} else {
    printf("Odd number");
}
```

Example: Prints "Even number" if x is even, otherwise "Odd number".



3. if-else if ladder

- Used to check multiple conditions one after another.

```
if (marks >= 90) {  
    printf("Grade A");  
} else if (marks >= 75) {  
    printf("Grade B");  
} else if (marks >= 50) {  
    printf("Grade C");  
} else {  
    printf("Fail");  
}
```

Example: Prints grade based on marks.

4. Nested if

- An if statement inside another if.

```
if (x > 0) {  
    if (x % 2 == 0) {  
        printf("Positive Even");  
    } else {  
        printf("Positive Odd");  
    }  
}
```

Example: Checks if number is positive, then further checks if it is even or odd.

5. switch statement

- Used to handle multiple cases based on a single variable.

```
switch (day) {  
    case 1: printf("Monday"); break;  
    case 2: printf("Tuesday"); break;  
    case 3: printf("Wednesday"); break;  
    default: printf("Invalid day");  
}
```

Example: Prints the day name based on the value of day.

Loop Control Instructions in C Programming

Definition

Loop control instructions are used to **repeat a block of code multiple times** until a certain condition is met. They help avoid writing repetitive code manually.



Alright, let's break this down clearly — one by one, with **definition** and **example** for each loop and loop-control statement in C.

Loops in C

1. while loop

Definition:

The `while` loop is an entry-controlled loop. The condition is checked first; if true, the loop body executes.

Example:

```
int i = 1;
while(i <= 5) {
    printf("%d ", i);
    i++;
}
```

Prints numbers 1 to 5.

2. do-while loop

Definition:

The `do-while` loop is an exit-controlled loop. The body executes at least once before the condition is checked.

Example:

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while(i <= 5);
```

Prints numbers 1 to 5, even if condition fails after first run.

3. for loop

Definition:

The `for` loop is entry-controlled and combines initialization, condition, and increment/decrement in one line.

Example:

```
for(int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```



Prints numbers 1 to 5.

Loop Control Statements

1. break

Definition:

Used to immediately exit from the loop, regardless of condition.

Example:

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3) break;  
    printf("%d ", i);  
}
```

Prints 1 2 and stops when `i == 3`.

2. continue

Definition:

Skips the current iteration and moves to the next iteration of the loop.

Example:

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3) continue;  
    printf("%d ", i);  
}
```

Prints 1 2 4 5 (skips 3).

Functions in C Language

Definition

A **function** in C is a named block of code that performs a specific task. Functions allow you to divide a program into smaller, reusable parts. This makes code easier to read, debug, and maintain. Every C program must have at least one function — `main()` — but you can define additional functions as needed.

Advantages of Functions



- **Reusability:** Write once, use multiple times.
 - **Modularity:** Break complex problems into smaller tasks.
 - **Readability:** Code becomes easier to understand.
 - **Maintainability:** Easier to update or fix parts of the program.
-

Types of Functions

1. Library Functions

- Predefined functions provided by C (e.g., `printf()`, `scanf()`, `sqrt()`).
- Declared in header files like `<stdio.h>`, `<math.h>`.

2. User-defined Functions

- Functions created by the programmer to perform specific tasks.
- Example:

```
int square(int x) {  
    return x * x;  
}
```

Syntax of a Function

```
return_type function_name(parameter_list) {  
    // body of the function  
}
```

- **return_type** → Type of value returned (e.g., `int`, `float`, `void`).
 - **function_name** → Identifier for the function.
 - **parameter_list** → Variables passed into the function.
 - **body** → Statements that define the task.
-

Function Components

1. Function Declaration (Prototype)

- Tells the compiler about the function's name, return type, and parameters.

```
int square(int x);
```

2. Function Definition

- Contains the actual code (logic).

```
int square(int x) {  
    return x * x;  
}
```

3. Function Call

- Executes the function.




```
int result = square(5);  
printf("Square is %d", result);
```

Key Points

- Functions improve **structure** and **clarity** of programs.
- `main()` is the starting point of every C program.
- Parameters allow passing values into functions.
- Functions can return values or be `void` (no return).
- Recursion is possible (a function calling itself).

Recursion in C Language

- **Definition:**
Recursion occurs when a function calls itself directly or indirectly. It solves problems by breaking them into smaller sub-problems.
- **Key Points:**
 - Every recursive function must have a **base case** (termination condition).
 - Recursive calls continue until the base case is satisfied.
 - Recursion uses the **stack** in memory.

Syntax

```
return_type function_name(parameters) {  
    if(base_condition) {  
        // stop recursion  
        return value;  
    } else {  
        // recursive call  
        return function_name(modified_parameters);  
    }  
}
```

Example 1: Factorial using Recursion

```
#include <stdio.h>  
  
int factorial(int n) {  
    if(n == 0 || n == 1)    // base case  
        return 1;  
    else  
        return n * factorial(n - 1); // recursive call  
}  
  
int main() {
```



```
int num = 5;
printf("Factorial of %d is %d", num, factorial(num));
return 0;
}
```

Output:

Factorial of 5 is 120

Example 2: Fibonacci Series using Recursion

```
#include <stdio.h>

int fibonacci(int n) {
    if(n == 0) return 0;
    if(n == 1) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}

int main() {
    int i;
    for(i = 0; i < 10; i++) {
        printf("%d ", fibonacci(i));
    }
    return 0;
}
```

Output:

0 1 1 2 3 5 8 13 21 34

Advantages

- Makes complex problems easier by dividing them into smaller parts.
- Very useful in mathematical problems (factorial, fibonacci, GCD, etc.).

Disadvantages

- Too many recursive calls may cause **stack overflow**.
 - Sometimes recursion is slower compared to iteration.
-

Practice Questions

1. Write a recursive function to calculate the sum of first n natural numbers.
2. Write a recursive function to find the GCD of two numbers.
3. Write a recursive function to print numbers from n down to 1.



